SecureClaw: Non-Bypassable, Privacy-Preserving Policy Enforcement for Agentic Tool Use and Skill Supply Chains

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agent tool-use security faces two coupled problems: compromised runtimes can bypass in-process guardrails at effect time, and centralized policy enforcement can itself become a privacy sink. We present SecureClaw, an execution control plane organized around three invariants: a cryptographically non-bypassable PREVIEW→COMMIT boundary, privacy-preserving outsourced policy checks via optional single-auditor privacy (SAP), and runtime plaintext blindness via handle-first secrets plus explicit declassification gates. Commit succeeds only when an independent executor verifies dual commit-proof shares bound to a canonical request commitment, delegation context, freshness, and replay state. SAP uses fixed-shape PIR+MPC routing with an explicit leakage contract and a simulation-based single-server transcript privacy claim. On official C1–C5 coverage, SecureClaw achieves 0.00% observed attack leakage and 100.00% attack blocking with 80.00% benign allow in our deterministic run. In single-policy-server distinguishability sweeps, shaping reduces separable signal (PIR MI 0.414 bits unshaped) to near-zero MI under unified fixed-shape mode. The artifact additionally provides a compromised-runtime bypass suite, delegated multi-principal tests, and protocolized extension designs for AP2 case-study, enterprise plaintext-proxy baselines, and multimodal stress benchmarks.

1 Introduction

LLM agents now trigger irreversible effects: sending messages, invoking external APIs, and handling payment-like workflows. Consider a finance assistant that can read payroll documents, email vendors, and call payment or webhook APIs. If attacker-controlled context reaches the runtime, prompt injection and tool-chain compromise are no longer only "reasoning errors"; they become execution-time security failures. Recent evidence shows that simple prompt injection can make tool-calling agents leak personal data observed during task execution [Alizadeh et al., 2025]. At the protocol layer, AP2-style agent payments further raise the stakes because manipulated agent actions can directly affect financial outcomes [AP2 Protocol, 2025, Debi and Zhu, 2026]. This shifts the core security question from "what should the model intend?" to "what can actually commit at the effect sink?"

A natural reaction is to add centralized policy control (AI gateway, DLP, LLM firewall). However, this introduces a second failure mode: the same control plane that enforces policy may observe and log sensitive query values. We call this the *control-plane privacy paradox*.

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

Thesis. Secure agent deployment needs an execution-time authorization boundary; enterprise deployment additionally needs (i) control-plane privacy and (ii) runtime plaintext blindness, otherwise the security layer becomes a privacy sink and the agent can still leak secrets from its own context.

Why prior work is insufficient. Runtime guardrails such as DRIFT and IPIGuard improve mediated behavior, but once the runtime itself is compromised they cannot force the final effect sink to reject an unauthorized commit. Execution-boundary designs such as Faramesh move enforcement to the right architectural place, but do not address the enterprise deployment gap created by centralized plaintext policy inspection or the case where the runtime already holds plaintext secrets. SecureClaw keeps the execution-boundary philosophy while adding these two missing invariants.

Contributions.

- **Pillar 1: Non-bypassable execution boundary.** We define and implement a PREVIEW→COMMIT protocol in which no external side effect commits unless the executor verifies dual commit-proof shares bound to a canonical request hash (including delegation context), freshness window, and replay state.

- **Pillar 2: Privacy-preserving policy outsourcing.** We formalize outsourced policy privacy with an explicit leakage contract $L_{\mathsf{policy}} = (L_{\mathsf{PIR}}, L_{\mathsf{MPC}}, L_{\mathsf{CONFIRM}}, L_{\mathsf{TIME}})$ and state a simulation-based single-auditor theorem: each single policy-server transcript is simulatable from L_{policy} under non-collusion and fixed-shape routing.

- **Pillar 3: Runtime plaintext blindness and multi-principal correctness.** We compose handle-first plaintext confinement, sanitize-at-commit integrity, capsule-conditional skill confinement, and delegation-aware hash-context binding into one system-level theorem with explicit assumption partitioning.

- **Evidence and claim-gating artifacts.** On official C1–C5 coverage, SecureClaw reaches 0.00% observed attack leakage and 100.00% attack blocking with 80.00% benign allow. In SAP distinguishability sweeps, shaping collapses single-server intent signal from separable (PIR MI 0.414 bits) to near-zero. We additionally expose versioned, machine-checkable accept-predicate and capsule-contract verifiers that enforce claim-gating and reduced-claim downgrade semantics.

2 Background and Motivation

2.1 Agent leakage surfaces

Agent pipelines expose multiple leakage channels: final responses, inter-agent messages, tool arguments/results, and memory interfaces. Prompt injection and indirect context contamination exploit these channels across a wide class of benchmarks and real tool-use settings [Greshake et al., 2023, Zhan et al., 2024, Debenedetti et al., 2024, Yagoubi et al., 2026].

2.2 The control-plane privacy paradox

Operationally, enterprises often centralize policy checks for safety, auditing, and compliance. Yet centralized control planes may inspect and retain prompt and tool payloads [Microsoft Learn, 2026, Cloudflare, 2026, Cloudflare Developers, 2026, Dynatrace Docs, 2026]. Thus, a defense layer can become a high-value privacy sink. SecureClaw addresses this paradox directly: policy outsourcing remains centralized in function, but single-server transcript visibility is leakage-contracted and simulatable.

3 Threat Model and Security Goals

3.1 Parties and trust assumptions

We model five roles: (1) adversarial runtime $\mathcal{A}$ (controls prompts, tool invocations, and skill code), (2) trusted gateway $\mathcal{G}$, (3) policy servers $\mathcal{P}_0, \mathcal{P}_1$, (4) trusted executor $\mathcal{X}$ (sole side-effect sink), and (5) user confirmation source $\mathcal{U}$.

For SAP, we assume non-collusion between $\mathcal{P}_0$ and $\mathcal{P}_1$ (single-auditor threat). If both collude, single-auditor query privacy is impossible in this model.

3.2 Goals and non-goals

Goals.

- **Effect integrity (NBE):** no commit without valid dual, request-bound proof shares.
- **Control-plane privacy (SAP, optional mode):** single policy-server transcript leaks only L_{policy}.
- **Confidentiality and context correctness:** plaintext is not exposed to $\mathcal{A}$ by default; commits remain bound to caller/session/principal/delegation context.

Non-goals.

- No claim if gateway or executor is compromised.
- No claim against kernel/hardware side channels.
- No SAP claim under full policy-server collusion.

4 Design Overview

SecureClaw separates planning from effects. Runtime logic may propose actions through a single high-level interface, but external side effects are committed only by the executor. The gateway mediates policy preview, handleization/declassification, delegation checks, and (in SAP mode) fixed-shape policy outsourcing.

At a high level:

1. Runtime calls `act(intent_id, inputs, constraints, caller)`.
2. Gateway canonicalizes and computes request binding hash.
3. Policy path returns dual commit-proof shares plus semantic bits.
4. Executor verifies accept predicate and either fail-closes or commits effect.

5 Non-Bypassable Effects Protocol

5.1 Canonical request binding

For effectful tuple $\rho = (\mathsf{intent_id}, \mathsf{caller}, \mathsf{session}, \mathsf{inputs}, \mathsf{hctx})$ with $\mathsf{hctx} = (\mathsf{external_principal}, \mathsf{delegation_jti})$, we define

$$\mathsf{ReqBind}(\rho) = \begin{cases} \text{HMAC-SHA256}(k_{\text{bind}}, \mathsf{CanonJSON}(\rho)) & \text{if keyed mode enabled,} \\ \text{SHA256}(\mathsf{CanonJSON}(\rho)) & \text{legacy compatibility mode.} \end{cases}$$

Both gateway and executor use the same canonical serializer and binding mode. The wire field name remains '`request_sha256`' for protocol compatibility.

5.2 PREVIEW→COMMIT and accept predicate

PREVIEW obtains dual proof shares from independent policy servers. COMMIT succeeds iff executor verifies all of:

1. two valid, distinct-server MAC-authenticated commit proofs,
2. freshness within TTL,
3. agreement on $(\mathsf{action_id}, \mathsf{program_id}, \mathsf{request_sha256})$,
4. recomputed request-hash equality (including hctx),
5. replay non-membership,
6. semantic constraints (allow/confirm/sanitize).

Any failure is fail-closed.

5.3 Security statement

Theorem 1 (Appendix A) reduces commit bypass to MAC forgery, hash collision/serialization inconsistency, or explicit operational assumption failures (freshness/replay-state assumptions).

6 Control-Plane Privacy Paradox and SAP

6.1 Leakage contract

We model single-server observable leakage as

$$L_{\mathsf{policy}} = (L_{\mathsf{PIR}}, L_{\mathsf{MPC}}, L_{\mathsf{CONFIRM}}, L_{\mathsf{TIME}}).$$

Allowed terms include fixed endpoint class, public program id/circuit shape, padded batch occupancy, and coarse timing buckets. Forbidden terms include true PIR indices and plaintext predicate inputs. Intent hiding is claimed only when public observables (program id, endpoint class, batch geometry) are constant across covered intents.

6.2 Single-auditor privacy game

Adversary chooses workloads (W_0, W_1) with identical leakage: $L_{\mathsf{policy}}(W_0) = L_{\mathsf{policy}}(W_1)$. Challenger samples $b \leftarrow \{0, 1\}$, executes W_b, and reveals one policy-server view $\mathsf{View}(\mathcal{P}_\sigma)$. Adversary outputs b'. SAP requires negligible distinguishing bias over 1/2.

6.3 Theorem and interpretation

Theorem 5 states that under single-server PIR privacy, HBC 2PC privacy, non-collusion, and fixed-shape scheduling, each single-server transcript is simulatable from L_{policy}. Interpretation: one policy server cannot recover sensitive query values beyond explicit allowed leakage.

SAP does *not* protect against: (i) full policy-server collusion, (ii) intentionally unshaped deployment where intent-correlated metadata is left observable, and (iii) leakage channels outside the policy transcript model.

7 Formal Security

We state one system-level proposition and prove it via component theorems (Appendix A).

System proposition. For deployed SecureClaw interfaces, no external side effect commits without dual request-bound commit evidence; plaintext release and cross-principal replay remain constrained to explicit interfaces and assumptions; and in SAP mode, single-server transcript leakage is bounded by L_{policy}.

Component theorems.

- **NBE** (Theorem 1): commit integrity under dual-proof, hash binding, freshness, replay assumptions.
- **SM/PEI/SCS/DAS** (Theorems 3, 4, 6, 7): confidentiality, sanitize-at-commit integrity, capsule-conditional confinement, and delegation-aware binding.
- **SAP** (Theorem 5, optional mode): single-auditor transcript privacy.

Composition bound. Let $\mathsf{Bad}_{\mathsf{NBE}}, \mathsf{Bad}_{\mathsf{SM}}, \mathsf{Bad}_{\mathsf{PEI}}, \mathsf{Bad}_{\mathsf{SCS}}, \mathsf{Bad}_{\mathsf{DAS}}, \mathsf{Bad}_{\mathsf{SAP}}$ denote property-specific bad events. Then (Theorem 2):

$$\Pr[\mathsf{Exp}^{\mathsf{SC}} = 1] \leq \sum_{E \in \{\mathsf{NBE},\mathsf{SM},\mathsf{PEI},\mathsf{SCS},\mathsf{DAS},\mathsf{SAP}\}} \Pr[\mathsf{Bad}_E].$$

No mutual exclusivity assumption is needed; overlap only affects tightness.

8 Implementation

SecureClaw is implemented as a gateway-executor architecture with machine-checkable protocol specs. Core implementation anchors include:

- canonical binding and hash computation,
- machine-checkable protocol contracts (versioned schema + semantic accept-predicate spec),
- dual-proof verification and replay guard in executor,
- fixed-shape PIR/MPC routing for SAP mode,
- capsule contract verifier with mandatory-assertion claim gating and downgrade semantics,
- formal harnesses for security games and model checks.

Artifact-level validators include: `scripts/security_game_nbe_check.py`, `formal/secureclaw_model_check.py`, and `scripts/validate_specs.py`.

9 Evaluation

9.1 RQs and threat-model split

We organize evaluation around four reviewer-facing questions and three extension protocols: does SecureClaw stop leaks, does SAP preserve privacy, does the design survive delegation, and is the overhead acceptable?

Threat Model A (honest runtime). Native guardrail baselines (Codex/OpenClaw and mediated DRIFT/IPIGuard/AgentArmor) are interpreted as runtime-behavior defenses where direct executor bypass is out of scope.

Threat Model B (compromised runtime). Runtime can attempt direct effect bypass, replay/context swapping, and exfil paths outside intended tool mediation. SecureClaw's NBE/SCS claims are evaluated here.

9.2 RQ1: official C1–C5 comparison

SecureClaw achieves zero observed leakage on the official attack set. Compared with SecureClaw, attack-leak differences are significant for Codex native ($p = 4.67 \times 10^{-4}$) and OpenClaw native ($p = 9.41 \times 10^{-4}$). This run evaluates $n_{\mathsf{attack}} = 744$ and $n_{\mathsf{benign}} = 2520$ official rows under a fixed manifest. To keep baseline fairness auditable, defense decisions exclude dataset-truth labels and payload oracle fields (for example, attack/benign tags and secret-token ground truth), and all systems share the same manifest and aggregation rules.

System	Attack leak	Attack block	Benign allow
SecureClaw (full)	0.00%	100.00%	80.00%
Codex native	1.61%	98.39%	100.00%
OpenClaw native	1.48%	98.52%	0.00%

Table 1: Official C1–C5 comparison under neutral native prompts. We report end-to-end runtime separately in RQ4 because cross-system latency is not apples-to-apples when the mediation stacks differ.

Mode	PIR MI (bits)	PIR MAP acc	MPC MI (bits)	MPC MAP acc
Unshaped	0.4143	0.514	–	–
Shaped (pad1, no cover)	0.0000	0.207	0.0000	0.278
Shaped (pad4, no cover)	0.0000	0.207	0.0000	0.288
Shaped (pad4, cover)	0.0000	0.331	0.0000	0.303

Table 2: Single-policy-server distinguishability sweep (3-way intent classes, chance = 1/3).

We intentionally keep latency out of Table 1: native baselines and SecureClaw exercise different mediation paths, so apples-to-apples timing belongs in RQ4, where we isolate authorization overhead from model-inference time.

Official AgentLeak coverage is limited to channels C1–C5. For channels C6/C7 (logs/audit and skill ingress), we report a separate synthetic full-channel suite: current SecureClaw snapshot records 0.00% attack leakage on C6/C7 with 100.00% benign allow for those channels.

9.3 RQ2: SAP transcript distinguishability

Unshaped routing exposes separable signal. Under unified fixed-shape routing, MI collapses to near zero and classifier accuracy approaches chance-level behavior. This is the deployment contrast that matters: a centralized plaintext proxy can enforce policy, but it trivially sees sensitive query values, whereas SAP asks what any single policy server can infer from its own transcript and drives that signal toward chance.

Query-value privacy sanity check. To separate *intent-shape* leakage from *query-value* leakage, we run a 16-way recipient-index task on single-server transcript features. Observed single-server recipient prediction remains at or below chance (chance = 6.25%; unshaped 2.89%, shaped-pad4-cover 4.50%) with near-zero MI. This is consistent with PIR single-server index privacy: shaping is primarily needed to suppress intent-correlated metadata rather than to hide PIR query indices.

9.4 RQ3: delegated multi-agent safety

In a deterministic delegated campaign (7 cases), all outcomes match expected policy semantics (7/7 pass), including missing-delegation denial, revocation denial, dual-principal denial, and mediated memory access. Gateway check latency remains low (median 0.38 ms, p95 5.56 ms).

9.5 RQ4: systems cost and fairness

We separate enforcement overhead from model-inference latency. Shaping overhead is decomposed into (i) padding/cover compute, (ii) mixer queueing, and (iii) transport framing. Executor verification remains constant-sized (dual MAC verify + replay lookup).

9.6 E1 (completed): compromised-runtime bypass suite

We evaluate direct bypass attempts: missing proof share, one-server proof only, request/session/caller/hctx mismatches, replay, and downgraded baselines. The current arti-

fact report (`compromised_bypass_report.json`) records 16/16 expected outcomes, including fail-closed denial for bypass attempts and explicit bypass observability in intentionally insecure baselines.

9.7 E2 (completed): single-auditor compromise simulation

The SAP sweep operationalizes a single-policy-server adversary using transcript-only features, MI, and classifier attacks. This directly instantiates the SAP game with empirical distinguishers under controlled shaping ablations.

9.8 E3–E6 (protocolized extension design)

We provide complete experiment protocols to close remaining external-validity gaps:

- **E3 AP2 case study** (agent payments): compare vanilla AP2, AP2 + prompt/DLP shields, and AP2 + SecureClaw (with/without SAP), measuring unauthorized mandate rate, sensitive-data leakage, utility, and latency [AP2 Protocol, 2025, Debi and Zhu, 2026].
- **E4 AgentDojo exfiltration extension**: reproduce data-flow exfiltration tasks and compare vanilla, shielded, and SecureClaw variants under the same task manifest [Alizadeh et al., 2025, Debenedetti et al., 2024].
- **E5 VPI-Bench subset**: stress-test computer-use scenarios where effects can be mediated by executor; unmediated UI side effects are reported explicitly as scope limits [Cao et al., 2025].
- **E6 enterprise DLP-proxy baseline**: compare centralized plaintext policy inspection versus SAP-shaped outsourcing, measuring single-auditor distinguishability and direct sensitive-value visibility in policy-plane logs.

These protocols are specified with baseline mapping and fairness constraints (threat-model alignment, no label-oracle defenses), and are included to avoid over-claiming beyond measured evidence.

10 Limitations and Discussion

SecureClaw still depends on trusted gateway and executor. SAP requires non-collusion between policy servers. SCS is conditional on a valid capsule mediation contract. If an agent can trigger unmediated OS/UI effects outside the executor path, NBE does not apply; such paths must be wrapped by a mediated executor to inherit the guarantee. If keyed request binding is disabled, legacy unkeyed request-hash observability can permit offline guessing on low-entropy fields; keyed mode mitigates this by using an HMAC commitment secret shared only by gateway and executor.

Threat-model mismatch is a practical review risk: native guardrails are meaningful in Threat Model A but are not designed to satisfy Threat Model B's non-bypassable requirements. We therefore separate these evaluations explicitly.

11 Related Work and Positioning

Prompt-injection attacks and tool-chain leakage in LLM agents are well established [Greshake et al., 2023, Zhan et al., 2024, Shi et al., 2025, Li et al., 2025b, Debenedetti et al., 2024, Yagoubi et al., 2026].

Runtime guardrails. DRIFT, IPIGuard, and AgentArmor improve mediated behavior inside the model/runtime loop, which is valuable under Threat Model A, but they do not by themselves instantiate an executor-side non-bypassable commit boundary [Li et al., 2025a, An et al., 2025, Wang et al., 2025].

Execution boundaries. Faramesh's AAB and SecureClaw share the core architectural idea that authorization must bind to execution rather than to advisory prompts alone [Fatmi, 2026]. Our claim is that this direction is still insufficient for enterprise deployment unless the control plane is privacy-preserving and the runtime is plaintext-blind by default.

Privacy-preserving outsourcing and dataflow confinement. SecureClaw therefore adds two invariants on top of the execution boundary: SAP limits what any single policy server can learn about decision inputs, and handle-first confinement limits what a compromised runtime can directly read and exfiltrate before commit. The key difference is not only "decision-before-execution," but also "who can learn the decision inputs" and "how cross-principal plaintext reuse is prevented."

12 Conclusion

SecureClaw is centered on one proposition: robust agentic execution security requires a non-bypassable commit boundary, and practical outsourced policy enforcement additionally requires control-plane privacy and runtime plaintext blindness. Our measured results support this three-pillar thesis on official benchmark coverage, single-auditor distinguishability tests, and delegated multi-principal checks. The provided E3–E6 protocols extend this line toward AP2, AgentDojo exfiltration workflows, computer-use stress settings, and enterprise control-plane privacy-sink baselines while preserving threat-model fairness.

References

Meysam Alizadeh, Zeynab Samei, Daria Stetsenko, and Fabrizio Gilardi. Simple prompt injection attacks can leak personal data observed by llm agents during task execution, 2025. URL `https://arxiv.org/abs/2506.01055`.

Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents, 2025. URL `https://arxiv.org/abs/2508.15310`.

AP2 Protocol. Ap2 protocol specification, 2025. URL `https://ap2-protocol.org/specification`. Accessed 2026-02-20.

Tri Cao, Bennett Lim, Yue Liu, Yuan Sui, Yuexin Li, Shumin Deng, Lin Lu, and Nay Oo. Vpi-bench: Visual prompt injection attacks for computer-use agents, 2025. URL `https://arxiv.org/abs/2506.02456`.

Cloudflare. Block unsafe prompts targeting your llm endpoints with firewall for ai, 2026. URL `https://blog.cloudflare.com/block-unsafe-llm-prompts-with-firewall-for-ai/`. Accessed 2026-02-24.

Cloudflare Developers. Data loss prevention (dlp) in cloudflare ai gateway, 2026. URL `https://developers.cloudflare.com/ai-gateway/features/dlp/`. Accessed 2026-02-24.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL `https://arxiv.org/abs/2406.13352`.

Tanusree Debi and Wentian Zhu. Whispers of wealth: Red-teaming google's agent payments protocol via prompt injection, 2026. URL `https://arxiv.org/abs/2601.22569`.

Dynatrace Docs. Mask sensitive data in logs, 2026. URL `https://docs.dynatrace.com/docs/analyze-explore-automate/logs/lma-use-cases/methods-of-masking-sensitive-data`. Accessed 2026-02-24.

Amjad Fatmi. Faramesh: A protocol-agnostic execution control plane for autonomous agent systems, 2026. URL `https://arxiv.org/abs/2601.17744`.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023. URL https://arxiv.org/abs/2302.12173.

Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. Drift: Dynamic rule-based defense with injection isolation for securing llm agents, 2025a. URL https://arxiv.org/abs/2506.12104.

Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Stac: When innocent tools form dangerous chains to jailbreak llm agents, 2025b. URL https://arxiv.org/abs/2509.25624.

Microsoft Learn. Set up logging for llm apis in azure api management, 2026. URL https://learn.microsoft.com/en-us/azure/api-management/api-management-howto-llm-logs. Accessed 2026-02-24.

Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents, 2025. URL https://arxiv.org/abs/2504.19793.

Peiran Wang, Yang Liu, Yunfei Lu, Yifeng Cai, Hongbo Chen, Qingyou Yang, Jie Zhang, Jue Hong, and Ye Wu. Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection, 2025. URL https://arxiv.org/abs/2508.01249.

Faouzi El Yagoubi, Ranwa Al Mallah, and Godwin Badu-Marfo. Agentleak: A full-stack benchmark for privacy leakage in multi-agent llm systems, 2026. URL https://arxiv.org/abs/2602.11510.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024. URL https://arxiv.org/abs/2403.02691.

A Appendix: Proof Chain for Non bypassable Effects and Composition

This appendix formalizes SecureClaw's core integrity claim, *no external side effect is committed without two independently authenticated commit proofs bound to the exact request and hash context*, and sketches composition with privacy and confinement components. The formal statements are aligned with the reference implementation and executable validation harnesses.

Scope and assumptions. We reason about a PPT adversary that fully controls the untrusted agent runtime and all skill code, and may send arbitrary requests to the gateway and the executor. The executor's MAC verification keys are not compromised. Policy servers are assumed honest for integrity and they compute commit proofs according to the protocol. Policy servers are also assumed honest but curious for privacy. Single server leakage is bounded by an explicit leakage function defined in `LEAKAGE_MODEL.md`.

Single proposition. The reviewable proposition for this paper is: no external side effect can be committed without dual valid request bound commit proofs, runtime plaintext release and patch bypass cannot occur outside explicit interfaces, and cross-principal proof reuse is rejected by hash-context binding. In optional SAP mode, a single policy server learns no more than the allowed leakage function under stated assumptions. The first clause is formalized by Theorem 1. The second and third clauses are formalized by Theorems 3, 4, and 7. The optional privacy clause is formalized by Theorem 5.

Assumption partition. Table 3 partitions assumptions by property so that composition dependencies are explicit.

Property	Security objective	Main assumptions
NBE	No unauthorized side effects	MAC EUF-CMA, hash consistency (+hctx), bounded time, replay ator
SM	No plaintext flow to runtime	Handle opacity with entropy bound, explicit declassification only
PEI	No commit-time patch bypass	Executor sanitize-before-effect semantics, deterministic patch derivatior
SCS	Skill bypass confinement	Capsule mediation contract and NBE plus SM
DAS	No cross-context proof reuse	Delegation signature validity, revocation semantics, hctx hash binding
SAP (optional)	Single server transcript privacy	PIR single server security, MPC HBC privacy, non-collusion

Table 3: Assumption partition for SecureClaw properties.

A.1 Cryptographic primitives and canonical binding

MAC. Let $\mathsf{MAC} = (\mathsf{Tag}, \mathsf{Vrfy})$ be a message authentication code with key space $\mathcal{K}$ and message space $\mathcal{M}$. We assume existential unforgeability under chosen message attack, abbreviated EUF CMA. The artifact uses HMAC-SHA256:

$$\mathsf{Tag}_K(m) := \text{HMAC-SHA256}(K, m), \quad \mathsf{Vrfy}_K(m, t) := [t = \mathsf{Tag}_K(m)].$$

Canonical serialization and request binding digest. Let $\mathsf{Canon}(\cdot)$ be a deterministic canonical JSON serialization implemented in `common/canonical.py`. Let

$$\mathsf{hctx} := (\mathsf{external_principal}, \mathsf{delegation_jti})$$

denote the hash context, serialized as a JSON object containing only present fields (empty object if both are absent). Define the request binding digest:

$$\mathsf{ReqBind}(\mathsf{intent_id}, \mathsf{caller}, \mathsf{session}, \mathsf{inputs}, \mathsf{hctx}) := \begin{cases} \text{HMAC-SHA256}(k_{\mathsf{bind}}, \mathsf{Canon}(\cdot)) & \text{if keyed mode enabled,} \\ \text{SHA256}(\mathsf{Canon}(\cdot)) & \text{legacy compatibility mode,} \end{cases}$$

where $\mathsf{Canon}(\cdot)$ expands to

$$\mathsf{Canon}\left(\begin{array}{l} \{\texttt{"v"} : 1,\ \texttt{"intent_id"} : \mathsf{intent_id},\ \texttt{"caller"} : \mathsf{caller},\ \texttt{"session"} : \mathsf{session}, \\ \ \texttt{"inputs"} : \mathsf{inputs},\ \texttt{"context"} : \mathsf{hctx}\} \end{array} \right).$$

We assume request-binding security for the configured mode (HMAC PRF-style in keyed mode, SHA-256 collision resistance in legacy mode) and *binding consistency*: both gateway and executor compute identical byte strings under $\mathsf{Canon}(\cdot)$ for semantically equal JSON objects. This is an engineering assumption enforced by using the same library routine in both places.

A.2 Commit proof syntax and executor acceptance predicate

Each policy server $\mathcal{P}_\sigma$ for $\sigma \in \{0, 1\}$ holds a MAC key K_σ and uses a key identifier field `kid` for rotation. Each policy server issues *commit proof shares*. A commit proof share is a pair $\pi_\sigma = (\mathsf{meta}_\sigma, t_\sigma)$, where:

$$\begin{aligned} \mathsf{meta}_\sigma := \{&\texttt{"v"} : 1,\ \texttt{"kind"} : \texttt{"commit"},\ \texttt{"server_id"} : \sigma,\ \texttt{"kid"} : \text{kid},\ \texttt{"ts"} : \tau, \\ &\texttt{"action_id"} : a,\ \texttt{"program_id"} : p,\ \texttt{"request_sha256"} : h, \\ &\texttt{"outputs"} : y_\sigma,\ \texttt{"commit_tag_share_b64"} : \text{tag}_\sigma\}, \end{aligned}$$

and $t_\sigma := \mathsf{Tag}_{K_\sigma}(\mathsf{Canon}(\mathsf{meta}_\sigma))$. The artifact enforces this format through a machine-checkable commit-evidence schema and a declarative acceptance-predicate specification used by executable validators.

Executor acceptance. Given an effect request req and two purported commit proof shares π_0, π_1, the executor $\mathcal{X}$ accepts if and only if the predicate $\mathsf{Accept}(\mathsf{req}, \pi_0, \pi_1)$ holds, where:

1. Both shares parse, have `v=1` and `kind="commit"`, and claim distinct `server_id` values in $\{0, 1\}$.
2. MAC verification succeeds: $\mathsf{Vrfy}_{K_\sigma}(\mathsf{Canon}(\mathsf{meta}_\sigma), t_\sigma) = 1$ for $\sigma \in \{0, 1\}$, using the key for the claimed `kid`.

3. Freshness: $|\text{now} - \tau| \leq \text{TTL}_{\mathsf{mac}}$ where $\text{TTL}_{\mathsf{mac}}$ is configured by the environment variable `POLICY_MAC_TTL_S`.
4. Context agreement: both shares bind the same (a, p, h) tuple.
5. Request binding: letting $(\mathsf{intent_id}, \mathsf{caller}, \mathsf{session}, \mathsf{inputs}, \mathsf{hctx})$ be the effectful fields of req, we require $h = \mathsf{ReqBind}(\mathsf{intent_id}, \mathsf{caller}, \mathsf{session}, \mathsf{inputs}, \mathsf{hctx})$.
6. Replay protection: the action identifier a has not been accepted before within the replay window. The artifact uses `_ReplayGuard.check_and_mark`.
7. Policy bits reconstructed from (y_0, y_1) indicate allow; if a confirmation bit is set then the request includes explicit user confirmation; and any sanitize patch is applied.

For freshness soundness we model bounded clock skew and bounded transport delay: if a share is minted at honest time t, executor acceptance is evaluated at time t' with $|t' - t| \leq \Delta_{\mathsf{clk}} + \Delta_{\mathsf{net}}$, and we require $\Delta_{\mathsf{clk}} + \Delta_{\mathsf{net}} < \text{TTL}_{\mathsf{mac}}$.

A.3 Security game for non bypassable effects

We formalize *non bypassable effects* as an integrity game where the adversary controls the untrusted runtime and can obtain valid commit proofs for adaptively chosen preview tuples, but must not be able to commit an effect outside the authorized set.

Oracles. The challenger provides a *commit oracle* $\mathcal{O}_{\mathsf{commit}}(\rho)$ that, on input $\rho = (\mathsf{intent_id}, \mathsf{caller}, \mathsf{session}, \mathsf{inputs}, \mathsf{hctx})$, returns a fresh action id a and valid proof shares (π_0, π_1) whose `request_sha256` equals $\mathsf{ReqHash}(\rho)$. The challenger records the authorized binding tuple $(a, p, \mathsf{ReqHash}(\rho), \rho)$ in a set $\mathcal{S}$.

Experiment $\mathsf{Exp}^{\mathsf{NBE}}(\lambda)$.

1. The challenger samples independent MAC keys $K_0, K_1 \leftarrow \mathcal{K}$ and initializes replay state empty.
2. The adversary $\mathcal{A}$ may adaptively query $\mathcal{O}_{\mathsf{commit}}(\rho)$.
3. $\mathcal{A}$ outputs an effect request $\mathsf{req}^\star$ and two shares $(\pi_0^\star, \pi_1^\star)$.
4. $\mathcal{A}$ wins if $\mathsf{Accept}(\mathsf{req}^\star, \pi_0^\star, \pi_1^\star) = 1$ and at least one of the following holds:
   (a) **No-auth:** the accepted $(a^\star, p^\star, h^\star)$ tuple is not in $\mathcal{S}$.
   (b) **Binding break:** $(a^\star, p^\star, h^\star)$ is in $\mathcal{S}$, but the request tuple $\rho(\mathsf{req}^\star)$ differs from the ρ recorded for that context.
   (c) **Replay:** the executor accepts the same $a^\star$ twice within the replay window.

A.4 Main theorem: no side effects without dual proofs

Theorem 1 (No side effects without dual commit proofs)**.** *Assume (i)* MAC *is EUF-CMA secure; (ii) SHA-256 is collision resistant and* $\mathsf{Canon}(\cdot)$ *is consistent; (iii) the executor enforces* Accept *as specified; (iv) bounded time drift/latency satisfies* $\Delta_{\mathsf{clk}} + \Delta_{\mathsf{net}} < \text{TTL}_{\mathsf{mac}}$*; (v) replay check-and-mark is atomic for each action id and replay state durability assumptions hold for the configured replay window. Then for any PPT adversary* $\mathcal{A}$ *there exist PPT reductions* $\mathcal{B}_0, \mathcal{B}_1, \mathcal{C}$ *such that:*

$$\Pr[\mathsf{Exp}^{\mathsf{NBE}}(\lambda) = 1] \leq \text{Adv}^{\text{euf-cma}}_{\mathsf{MAC}}(\mathcal{B}_0) + \text{Adv}^{\text{euf-cma}}_{\mathsf{MAC}}(\mathcal{B}_1) + \text{Adv}^{\text{coll}}_{\text{SHA256}}(\mathcal{C}) + \varepsilon_{\text{time}}(\lambda) + \varepsilon_{\text{replay_atomic}}(\lambda) + \varepsilon_{\text{replay_persist}}(\lambda),$$

where $\varepsilon_{\text{time}}$ *captures only violations of the time-drift/latency assumption,* $\varepsilon_{\text{replay_atomic}}$ *captures failures of atomic check-and-mark semantics, and* $\varepsilon_{\text{replay_persist}}$ *captures replay-state durability violations (for example restart/fallback configurations outside the claimed durable window).*

Proof (reduction by cases). Fix an adversary $\mathcal{A}$ that wins the experiment and consider the accepting transcript $(\mathsf{req}^\star, \pi_0^\star, \pi_1^\star)$. By acceptance, both MACs verify and both meta records bind the same context $(a^\star, p^\star, h^\star)$ with fresh timestamps, and $h^\star$ equals the executor's recomputed $\mathsf{ReqHash}(\rho(\mathsf{req}^\star))$.

Case 1 (No-auth). Suppose $(a^\star, p^\star, h^\star) \notin \mathcal{S}$. The challenger never queried its own commit oracle on the message $\mathsf{Canon}(\mathsf{meta}_0^\star)$ or $\mathsf{Canon}(\mathsf{meta}_1^\star)$ for this context. Since both tags verify, at least one share is a fresh valid tag on a message not previously tagged under the corresponding key. We build a UF-CMA forger $\mathcal{B}_0$ (symmetrically $\mathcal{B}_1$) that embeds its UF-CMA challenge key as K_0, answers $\mathcal{O}_{\mathsf{commit}}$ by querying its signing oracle for $\mathsf{Canon}(\mathsf{meta}_0)$, and outputs $(\mathsf{Canon}(\mathsf{meta}_0^\star), t_0^\star)$ when $\mathcal{A}$ wins in this case.

Case 2 (Binding break). Suppose $(a^\star, p^\star, h^\star) \in \mathcal{S}$, but $\rho(\mathsf{req}^\star) \neq \rho$ recorded for that context. Acceptance implies:

$$h^\star = \mathsf{ReqHash}(\rho(\mathsf{req}^\star)) = \mathsf{ReqHash}(\rho).$$

If $\rho(\mathsf{req}^\star) \neq \rho$ then we have found a collision in SHA-256 *under the canonical encoding* (or an inconsistency in Canon). A reduction $\mathcal{C}$ outputs the two distinct canonical inputs whose hashes are equal. This contradicts collision resistance or the engineering consistency assumption.

Case 3 (Replay). If the executor accepts a previously accepted $a^\star$ within the replay window, then the replay guard's check-and-mark failed due to loss/corruption of replay state or lack of atomicity. This is outside the cryptographic model and is captured by $\varepsilon_{\text{replay_atomic}} + \varepsilon_{\text{replay_persist}}$.

Case 4 (Time-bound assumption failure). If acceptance occurs outside the intended operational freshness envelope due to unmodeled clock drift or transport delay violations, this contributes only to $\varepsilon_{\text{time}}$.

Taking a union bound over the cases yields the theorem. □

A.5 Session, caller, TTL binding

Theorem 1 already incorporates binding of caller and $\mathsf{session}$ because they are included in $\mathsf{ReqHash}$ together with hctx. Freshness (TTL) is enforced by the executor's timestamp check under the stated $\Delta_{\mathsf{clk}}, \Delta_{\mathsf{net}}$ assumption. Thus, any attempt to reuse a commit proof for a different caller, session, hash-context binding, or effectful input must either fail MAC verification/binding checks or imply a hash collision. Replay claims across restarts additionally require either persistent replay state (`EXECUTOR_REPLAY_DB_PATH`) or an explicit no-restart assumption over the replay window.

A.6 Composition with privacy and confinement

We state a composition theorem that makes the conjunction of properties explicit. For mechanized consistency, the artifact includes an explicit state model checker `formal/secureclaw_model_check.py` that enumerates bounded adversary behaviors under idealized cryptography and checks bounded consistency for NBE, SM, PEI, DAS, and SCS shape assumptions when the executor enforces Accept and the interface fields match the schema.

Dependency decomposition. The combined argument separates responsibilities at interface boundaries. PIR and two party computation are used only for transcript privacy and do not enforce side effects. Executor acceptance and replay logic enforce effect integrity. Gateway handleization enforces runtime plaintext confinement. Capsule mediation enforces skill process and network confinement. This partition prevents hidden circular assumptions in the composition theorem.

Combined experiment. Define a combined experiment $\mathsf{Exp}^{\mathsf{SC}}(\lambda)$ for SecureClaw. The adversary controls the untrusted runtime and all skills and may send arbitrary requests to the gateway and the executor. The adversary may also corrupt at most one policy server

and obtain its full internal view. The experiment outputs 1 if any of the following bad events occurs.

- $\mathsf{Bad_{NBE}}$: an external side effect is committed without two valid commit proofs bound to the exact request hash, within the freshness window, and not replayed.
- $\mathsf{Bad_{SM}}$: the adversary learns a sensitive plaintext value that is not in the explicit declassification leakage L_{SM}.
- $\mathsf{Bad_{PEI}}$: the executor accepts an effect commit where a required sanitize patch was not applied to the committed payload.
- $\mathsf{Bad_{SCS}}$: while the capsule mediation contract holds, the adversary performs a forbidden direct action that bypasses the gateway and executor.
- $\mathsf{Bad_{DAS}}$: commit evidence minted for one $(\mathsf{caller}, \mathsf{session}, \mathsf{hctx})$ is accepted under a different binding context.
- $\mathsf{Bad_{SAP}}$: the adversary distinguishes two action sequences with equal leakage under L_{policy} by observing the full transcript view of one policy server (optional mode).

Theorem 2 (Composition of SecureClaw properties)**.** *Assume the conditions of Theorems 1, 3, 4, 6, and 7. If SAP mode is claimed, additionally assume Theorem 5. Then for any probabilistic polynomial time adversary $\mathcal{A}$ the winning probability in the combined experiment satisfies*

$$\Pr[\mathsf{Exp}^{\mathsf{SC}}(\lambda) = 1] \leq \Pr[\mathsf{Bad_{NBE}}] + \Pr[\mathsf{Bad_{SM}}] + \Pr[\mathsf{Bad_{PEI}}] + \Pr[\mathsf{Bad_{SCS}}] + \Pr[\mathsf{Bad_{DAS}}] + \Pr[\mathsf{Bad_{SAP}}].$$

Moreover, each term is negligible in λ under the corresponding theorem assumptions.

Proof. The combined experiment outputs 1 if and only if at least one bad event occurs. By the union bound,

$$\Pr[\mathsf{Bad_{NBE}} \vee \mathsf{Bad_{SM}} \vee \mathsf{Bad_{PEI}} \vee \mathsf{Bad_{SCS}} \vee \mathsf{Bad_{DAS}} \vee \mathsf{Bad_{SAP}}] \leq \Pr[\mathsf{Bad_{NBE}}] + \Pr[\mathsf{Bad_{SM}}] + \Pr[\mathsf{Bad_{PEI}}] + \Pr[\mathsf{Bad_{SCS}}] + \Pr[\mathsf{Bad_{DAS}}]$$

No disjointness assumption is required for this step; event overlap only affects bound tightness. Theorem 1 bounds $\Pr[\mathsf{Bad_{NBE}}]$ by MAC unforgeability, hash collision resistance, bounded time assumptions, and replay-store atomicity/durability assumptions. Theorem 3 bounds $\Pr[\mathsf{Bad_{SM}}]$ by the handleization and declassification assumptions. Theorem 4 bounds $\Pr[\mathsf{Bad_{PEI}}]$ by executor patch-application soundness. Theorem 7 bounds $\Pr[\mathsf{Bad_{DAS}}]$ by delegation verification and hash-context binding. Theorem 5 bounds $\Pr[\mathsf{Bad_{SAP}}]$ by PIR and two party computation security up to the stated leakage. Theorem 6 bounds $\Pr[\mathsf{Bad_{SCS}}]$ by the capsule mediation contract and the integrity and confidentiality properties of the gateway and executor path. □

A.7 Capsule mediation contract as a system assumption

We define the capsule mediation contract as an explicit system assumption that can be validated by platform specific configurations and tests.

Contract definition. Let MC be a predicate over the runtime environment and its execution. The contract requires the following properties.

- Filesystem confinement. The runtime can read only its code and a dedicated workspace directory. The runtime cannot read host secret paths such as SSH keys or application credential stores.
- Network confinement. The runtime cannot initiate public network connections. If a network stack is used for transport then only the gateway address is reachable.
- Process confinement. The runtime cannot spawn arbitrary processes. Only a fixed allowlist required for the runtime operation is permitted.
- Transport confinement. The runtime can reach the gateway only through a single constrained transport endpoint, preferably a Unix domain socket.

Cross platform minimal realizations. The artifact provides a macOS capsule policy using `sandbox-exec` with profile `capsule/capsule.sb`, and a Linux capsule policy using `bubblewrap` with a network namespace in `capsule/run_smoke_linux.sh`. Both realizations use a constrained gateway transport and are exercised by the same smoke test logic in `capsule/smoke.py`.

Testable assertions. The smoke test asserts that direct reads of a host secret path fail, that direct public network requests fail, that direct HTTP posts to a local exfiltration endpoint fail when the capsule is configured for a netless transport, that direct process execution is denied in the macOS capsule configuration, and that a gateway action request succeeds through the constrained transport. These assertions are recorded as a structured JSON report.

A.8 Executable validation of the acceptance predicate

Theorem 1 is a cryptographic statement about any adversary in the model. The artifact also provides an executable specification that checks the concrete acceptance predicate.

Validation statement. Let Accept be the acceptance predicate specified in Section A.2. The artifact provides:

- unit-level checks that assert accept/reject behavior under mutated evidence;
- an end-to-end harness that exercises gateway, policy servers, and executor against adversarial request mutations.

The harness checks rejection under replay, session mismatch, caller mismatch, missing proof share, MAC mutation, request hash mutation, action identifier mutation, and timestamp expiration. These checks provide code level evidence that the artifact enforces the exact acceptance predicate assumed by the reduction in Theorem 1.

A.9 Secret myopia game and theorem

We formalize secret myopia as a simulation based confidentiality property for the untrusted runtime. The definition is implementation oriented. It treats the gateway and executor as trusted and asserts that sensitive plaintext does not flow to the untrusted runtime except through an explicit declassification interface.

Leakage function for SM. Let L_{SM} include: handle identifiers, handle labels, handle time to live values, policy decision bits, sanitization patch identifiers, and any explicitly declassified outputs that follow user confirmation. It excludes any other function of sensitive plaintext stored inside the gateway.

Experiment $\mathsf{Exp}^{\mathsf{SM}}(\lambda)$. The challenger initializes the gateway and gives the adversary control of the untrusted runtime. The adversary interacts with the gateway by issuing arbitrary action requests and receiving gateway responses. The adversary wins if it outputs a bit indicating which of two challenger chosen secret values was stored in a handle, with advantage non negligible over one half, without an allowed declassification event that reveals that value.

Theorem.

Theorem 3 (Secret myopia by handleization)**.** *Assume the gateway never returns sensitive plaintext to the untrusted runtime except through an explicit declassification interface that requires confirmation and includes declassified content in L_{SM}. Assume handle identifiers are sampled independently of plaintext with at least 128 bits of entropy (artifact default: `secrets.token_urlsafe(16)`), so any adversary making at most q_h online handle guesses in the valid window has success probability at most $q_h/2^{128}$. Then the system satisfies secret myopia with leakage L_{SM} up to additive $q_h/2^{128}$ and explicit declassification leakage.*

Proof. We construct a simulator $\mathcal{S}$ that, given L_{SM}, produces a distribution indistinguishable from the view of the adversary. Whenever the real gateway would mint a handle bound to some plaintext, $\mathcal{S}$ instead mints a fresh random handle identifier and records only the public metadata required by L_{SM}. All subsequent uses of that handle by the adversary are simulated by lookup on the simulator side. Because handle identifiers are sampled independently of plaintext and the adversary never receives plaintext unless it appears in the leakage, the simulated responses match the real responses distribution up to handle guessing probability $q_h/2^{128}$ and the declassified outputs already included in L_{SM}. Therefore any additional adversary advantage implies either a violation of the stated handle-entropy assumption or a direct plaintext flow outside the declassification interface, which contradicts the premise. □

Executable evidence. The artifact includes unit tests for handleization and declassification semantics across tool, inter-agent, and memory surfaces.

A.10 Patch-carrying egress integrity theorem

Experiment $\mathsf{Exp}^{\mathsf{PEI}}(\lambda)$. The adversary outputs a commit request and dual proof shares. The adversary wins if executor acceptance succeeds but the committed effect payload does not match the payload obtained by applying the required sanitize patch encoded by policy outputs.

Theorem.

Theorem 4 (Patch-carrying egress integrity)**.** *Assume the executor acceptance predicate is implemented as specified and all effect sinks execute only after executor-side sanitize application when policy requires patching. Then* $\Pr[\mathsf{Bad}_{\mathsf{PEI}}]$ *is bounded by implementation-spec divergence probability plus negligible cryptographic failure terms inherited from Theorem 1.*

Proof sketch. If acceptance succeeds, the executor has reconstructed policy bits and must apply required patch constraints before effect execution. Therefore any accepted unsanitized commit implies either a direct violation of executor implementation logic or a mismatch between implementation and declared acceptance semantics. □

A.11 Single auditor privacy game and theorem

We formalize single auditor privacy as transcript indistinguishability for a single policy server, parameterized by an explicit leakage function. The leakage function is defined in Appendix B.

Experiment $\mathsf{Exp}^{\mathsf{SAP}}(\lambda)$. The adversary outputs two action sequences W_0 and W_1 such that $L_{\mathsf{policy}}(W_0) = L_{\mathsf{policy}}(W_1)$. The challenger samples a bit b and executes the system on W_b. The challenger then reveals the full transcript view $\mathsf{View}(\mathcal{P}_\sigma)$ for one policy server $\mathcal{P}_\sigma$. The adversary outputs a guess b'.

Lemma 1 (PIR-view simulation). For fixed $\sigma \in \{0,1\}$, under single-server security of the DPF/FSS PIR scheme, there exists a PPT simulator $\mathsf{Sim}_\sigma^{\mathsf{PIR}}$ such that the PIR transcript fragment visible to $\mathcal{P}_\sigma$ is computationally indistinguishable from $\mathsf{Sim}_\sigma^{\mathsf{PIR}}(L_{\mathsf{PIR}})$.

Lemma 2 (MPC-view simulation). Under honest-but-curious security of the two-party MPC protocol, there exists a PPT simulator $\mathsf{Sim}_\sigma^{\mathsf{MPC}}$ such that the MPC transcript fragment visible to $\mathcal{P}_\sigma$ is computationally indistinguishable from $\mathsf{Sim}_\sigma^{\mathsf{MPC}}(L_{\mathsf{MPC}})$.

Lemma 3 (Deterministic fixed-shape composition). Assume the gateway scheduler is fixed-shape for covered intents: endpoint class, program id, batch geometry, and timing bucket policy are deterministic functions of L_{CONFIRM} and L_{TIME}. Then non-cryptographic transcript components are exactly computable from $(L_{\mathsf{CONFIRM}}, L_{\mathsf{TIME}})$.

Theorem.

Theorem 5 (Single-auditor transcript privacy under PIR+MPC)**.** *Assume:*

1. *DPF/FSS two-server PIR is secure against one server (single key-share view),*
2. *the fixed policy MPC is secure in the honest-but-curious model,*
3. *policy servers do not collude, and*
4. *deployment enforces fixed-shape routing/scheduling consistent with* $L_{\mathsf{policy}} = (L_{\mathsf{PIR}}, L_{\mathsf{MPC}}, L_{\mathsf{CONFIRM}}, L_{\mathsf{TIME}})$.

Then for each $\sigma \in \{0,1\}$ *there exists a PPT simulator* Sim_σ *such that for all PPT distinguishers* D*,*

$$\left|\Pr[D(\mathsf{View}(\mathcal{P}_\sigma)) = 1] - \Pr[D(\mathsf{Sim}_\sigma(L_{\mathsf{policy}})) = 1]\right|$$

is negligible in λ*.*

Proof. Fix σ and define hybrids. H_0 is the real single-server transcript. H_1 replaces PIR-visible fragments using $\mathsf{Sim}_\sigma^{\mathsf{PIR}}$; by Lemma 1, $H_0 \approx_c H_1$. H_2 additionally replaces MPC-visible fragments using $\mathsf{Sim}_\sigma^{\mathsf{MPC}}$; by Lemma 2, $H_1 \approx_c H_2$. By Lemma 3, all remaining scheduler/confirmation/timing metadata in H_2 is computable from $(L_{\mathsf{CONFIRM}}, L_{\mathsf{TIME}})$, hence from L_{policy}. Therefore H_2 is exactly (or efficiently) sampleable by a simulator $\mathsf{Sim}_\sigma(L_{\mathsf{policy}})$. Triangle inequality over hybrid distances gives negligible distinguishing advantage, proving the theorem. □

Executable evidence. The artifact includes fixed-shape mixing and padding logic for PIR and unified policy routing, plus end-to-end distinguishability sweeps that measure residual transcript signal under ablations.

A.12 Skill confined safety game and theorem

Skill confined safety is a conditional property. It relies on a capsule mediation contract that constrains the untrusted runtime environment. The contract is specified in Section 10.

Experiment $\mathsf{Exp}^{\mathsf{SCS}}(\lambda)$**.** The adversary controls the skill code and attempts to perform one of the following forbidden direct actions without using the gateway and executor path: host secret reads, public network exfiltration, or arbitrary process execution. The adversary wins if it succeeds while the capsule mediation contract holds.

Theorem.

Theorem 6 (Skill confined safety under a capsule mediation contract)**.** *Assume the capsule mediation contract holds and prevents the forbidden direct actions. Assume non bypassable effects and secret myopia hold for the gateway and executor path. Then the system satisfies skill confined safety.*

Proof. By the capsule mediation contract, any forbidden direct action attempt that does not traverse the gateway fails and cannot reach an external sink. Therefore any external side effect must traverse the executor, which enforces non bypassable effects. Any attempted exfiltration of sensitive plaintext through allowed interfaces is prevented by secret myopia unless the user explicitly declassifies it, in which case the leakage is intentional and auditable. Thus the adversary cannot perform forbidden direct actions or unauthorized effects under the stated assumptions. □

Executable evidence. The artifact provides a capsule smoke test that attempts direct filesystem reads, direct network access, and direct process execution from within the capsule boundary and asserts denial.

A.13 Delegation-aware binding safety theorem

Experiment $\mathsf{Exp}^{\mathsf{DAS}}(\lambda)$. The adversary first obtains valid PREVIEW evidence for one binding context $(\mathsf{caller}, \mathsf{session}, \mathsf{hctx})$ and then attempts COMMIT under a different context by swapping caller, session, external principal, or delegation identifier. The adversary wins if executor acceptance succeeds under the mismatched binding.

Theorem.

Theorem 7 (Delegation-aware hash-context binding safety)**.** *Assume delegation signatures are unforgeable, revocation checks are sound, canonicalization is consistent, and executor recomputes request binding digest including* hctx*. Then successful cross-context replay implies either MAC/hash break events already covered by Theorem 1 or signature/revocation assumption failure, hence* $\Pr[\mathsf{Bad}_{\mathsf{DAS}}]$ *is negligible under these assumptions.*

Proof sketch. Changing caller, session, or hash-context changes the bound request digest. Executor acceptance requires equality between recomputed hash and proof hash, so mismatched-context reuse fails unless cryptographic binding assumptions are violated. □

B Appendix: Leakage model and implementation mapping

This appendix defines the explicit leakage functions used by our single auditor privacy claim and by our end to end channel analysis. The definitions are artifact faithful and correspond to the implementation in `LEAKAGE_MODEL.md`.

B.1 Notation

Let λ be the security parameter. Let $\mathcal{G}$ be the gateway, let $\mathcal{P}_0$ and $\mathcal{P}_1$ be the policy servers, and let $\mathcal{X}$ be the executor. Let an action be identified by an action identifier a. Let the policy program identifier be p. Let the request binding digest be $h := \mathsf{ReqBind}$ as defined in Appendix A.1.

We write $\mathsf{View}(\mathcal{P}_\sigma)$ for the full transcript observed by policy server $\mathcal{P}_\sigma$, including request paths, message sizes, coarse timing buckets, and application layer fields.

B.2 Policy server leakage function L_{policy}

Single auditor privacy is defined relative to a leakage function that captures the intentionally revealed metadata to one policy server. We define

$$L_{\mathsf{policy}} := \langle L_{\mathsf{pir}},\ L_{\mathsf{mpc}},\ L_{\mathsf{confirm}},\ L_{\mathsf{time}} \rangle.$$

B.2.1 PIR leakage L_{pir}

The gateway queries membership databases using two server PIR based on DPF and FSS. A single policy server observes a key share per query and cannot infer the queried index under the DPF security assumption.

We capture the remaining metadata leakage as follows.

Public configuration. The domain size N and the fixed number of keys per subrequest are public. Bundling configuration is public.

Constant shape schedule. If mixing is enabled then the gateway sends exactly $\mathsf{pad_to}$ subrequests per tick and may fill slack with dummy subrequests. A single policy server learns the tick index and the fixed number of subrequests per tick.

Database name exposure. If bundling is enabled then the policy server sees a single bundle database name for all membership checks. Otherwise the policy server sees the database name per request.

Therefore we define L_{pir} as the tuple containing the public domain size, the fixed keys per subrequest, the observed database name, the tick schedule parameters, and the multiset of action identifiers included in each tick.

B.2.2 Two party computation leakage L_{mpc}

The policy program is evaluated using two party computation over XOR shared inputs. We assume honest but curious privacy for the two party computation protocol. Intent-category hiding at this layer requires that observable MPC metadata be fixed across covered intents, in particular a constant `program_id`, constant endpoint class, and constant batch shape parameters.

We define L_{mpc} as the tuple containing the program identifier, the circuit shape metadata which is constant in unified mode, the tick schedule parameters, and the multiset of action identifiers included in each tick. If `program_id` or endpoint class varies by intent, that variation is part of L_{mpc} and intent-category hiding is not claimed for those deployments.

B.2.3 Confirm path leakage L_{confirm}

When confirm path scanning is enabled the gateway may execute block PIR queries for a deterministic finite automaton transition table. If the confirm path is invoked conditionally then a single policy server may learn whether a scan occurred. If step padding is disabled then a single policy server may learn the number of steps.

We define L_{confirm} to include the scan invocation bit, the number of scan steps after any configured padding, the public block size, and the public transition table parameters. Concretely, with `PAD_DFA_STEPS=1` and cap `MAX_DFA_SCAN_CHARS`=M, the step count term becomes deterministic ($n_{\mathsf{confirm}} = M$); otherwise n_{confirm} may reveal message-length-correlated shape and is explicitly leaked.

B.2.4 Timing leakage L_{time}

The policy server can observe coarse arrival times and response times. We do not model kernel side channels. With mixers enabled we define per-layer bucket functions:

$$b_{\mathsf{pir}}(t) := \lfloor t/\Delta_{\mathsf{pir}} \rfloor \,, \quad b_{\mathsf{mpc}}(t) := \lfloor t/\Delta_{\mathsf{mpc}} \rfloor \,,$$

where $\Delta_{\mathsf{pir}} = \texttt{PIR_MIX_INTERVAL_MS}/1000$ and $\Delta_{\mathsf{mpc}} = \texttt{MPC_MIX_INTERVAL_MS}/1000$. Then L_{time} includes the bucket indices, tick counts per window, and public scheduler configuration. When a mixer is disabled, fine-grained per-request timing of that layer is treated as additional leakage and must be declared outside the fixed-shape claim.

B.3 System leakage function L_{sys}

We define application level leakage across seven channels inspired by AgentLeak. For an execution trace we define

$$L_{\mathsf{sys}} := \langle L_{C1}, L_{C2}, L_{C3}, L_{C4}, L_{C5}, L_{C6}, L_{C7} \rangle.$$

B.3.1 Channel definitions

$C1$ **final output.** L_{C1} includes the final decision type, a coarse output length, a reason code, and a timing bucket. It excludes sensitive plaintext unless explicitly declassified.

$C2$ **inter agent messages.** L_{C2} includes message counts, coarse timing, and handle identifiers used as payload references. It excludes sensitive plaintext because payloads are handleized.

$C3$ **tool inputs.** L_{C3} includes the intent identifier or a shadow class identifier, the set of input key names, a reason code, and a timing bucket. It excludes sensitive plaintext because sensitive fields are represented as handles.

Channel	Primary implementation path	Primary controls
$C1$	`gateway/mcp_server.py`, `gateway/egress_policy.py`	reason codes, sanitize patch,
$C2$	`gateway/router.py`, `gateway/handles.py`	handle bindings, leakage budg
$C3$	`gateway/mcp_server.py`, `gateway/policy_unified.py`	unified routing, bundle selecti
$C4$	`gateway/executors/*`	output handleization, declassi
$C5$	`gateway/executors/memoryexec.py`	memory handle mediation, le
$C6$	`gateway/audit.py`, `executor_server/server.py`	structured metadata logging,
$C7$	`gateway/policy_unified.py`, `gateway/capabilities.yaml`, `capsule/smoke.py`	skill IOC checks, caller capab

Table 4: Per-channel leakage mapping to implementation and controls.

$C4$ **tool outputs.** L_{C4} includes a tool status code, a coarse output summary length, handle identifiers returned as artifacts, and a timing bucket. It excludes sensitive plaintext unless explicitly declassified.

$C5$ **memory.** L_{C5} includes the memory namespace, a key identifier, a record count, handle identifiers stored, and a timing bucket. It excludes plaintext memory values because memory stores handle records.

$C6$ **logs and audit.** L_{C6} includes audit event type, caller identity, session identity, reason codes, and hash chain metadata. It excludes plaintext secrets.

$C7$ **skill supply chain ingress.** L_{C7} includes a skill digest, membership hit bits for known bad indicators, an install marker bit, a reason code, and a timing bucket. It excludes raw skill content unless explicitly declassified.

B.3.2 Channel to implementation mapping

Table 4 maps each channel leakage function to the implementation path and primary controls.

For $C7$, the capsule-mediation anchors are `capsule/MC_CONTRACT_SPEC.md`, `spec/secureclaw_capsule_contract_v1.json`, `capsule/smoke.py`, and `capsule/verify_contract.py`.

B.4 Residual leakage and allowed leakage items

We list the allowed leakage items and their sources. Each item is either required for correctness or explicitly accepted as leakage.

- Action identifier exposure to policy servers through fixed shape batching.
- Program identifier exposure to policy servers through unified routing.
- Tick schedule parameters such as interval and pad size.
- Coarse timing buckets at tick granularity.
- Audit metadata such as reason codes and hash chain pointers.

We treat the following as residual risks that are not eliminated by default configuration.

- Conditional confirm path invocation may leak a branch bit.
- Legacy-mode residual only: if keyed request binding is disabled, low-entropy guessing attacks against observable request digests are possible in principle when an attacker can enumerate candidates. With conditional min-entropy k over candidate inputs and q_d offline trials, residual dictionary advantage is upper-bounded by

$$\varepsilon_{\mathsf{dict}} \leq \min(1, q_d/2^k).$$

Control	Code path	Configuration
Request binding hash	`common/canonical.py`	none
Commit proof verify	`executor_server/server.py`	`POLICY_MAC_TTL_S`
Unified routing and program id	`gateway/policy_unified.py`	`UNIFIED_POLICY`, `MIRAGE_POLICY_PROGRAM_ID`
PIR bundling	`gateway/fss_pir.py`	`USE_POLICY_BUNDLE`
Oblivious bundle selection	`policy_server/build_dbs.py`	`POLICY_BUNDLE_NAME`
PIR mixing and padding	`gateway/fss_pir.py`	`PIR_MIX_ENABLED`, `PIR_MIX_PAD_TO`
PIR cover traffic	`gateway/fss_pir.py`	`PIR_COVER_TRAFFIC`
MPC mixing and padding	`gateway/policy_unified.py`	`MPC_MIX_ENABLED`, `MPC_MIX_PAD_TO`
MPC cover traffic	`gateway/policy_unified.py`	`MPC_COVER_TRAFFIC`
Confirm path DFA scan	`gateway/guardrails.py`	`DLP_MODE`, `PAD_DFA_STEPS`
Leakage budgets	`gateway/leakage_budget.py`	`LEAKAGE_BUDGET_*`

Table 5: Leakage controls and their implementation mapping.

B.5 Evaluation coverage note for channels

Official AgentLeak cases cover channels $C1$ through $C5$ only. Channels $C6$ and $C7$ are evaluated with the artifact synthetic full-channel suite, with explicit evidence index and reproduction commands in `LEAKAGE_EVIDENCE.md`.

B.6 Mapping to implementation and configuration

Table 5 maps the major leakage controls to code paths and configuration keys.